BAGIRA

Technical Test – Full-Stack developer (Backend oriented)

Dear candidate,

As part of the recruitment process at Bagira Systems, you are required to complete this assignment. You will receive the assignment at the time of your choice (e.g., 9 AM) and must submit your solution within 24 hours (e.g., by 9 AM the next day) ***please read the description carefully before starting.***

Objectives:

Implement missing parts in a provided code skeleton (backend and frontend) for a small web application used to create training scenarios and manage entities inside each scenario

Instructions:

1. Clone this repository – Bagira-Assignment and create a fork with the following structure – 'BagiraAssignment_<YourName>'.
2. Make sure to work on the folder named **'Backend-Oriented'**.
3. A minimal backend starter is provided in the `backend/` folder - Build your backend from there.
4. Choose either the React or Angular frontend skeleton in the `frontend/` folder and connect your backend to it. Specify which one you chose.
5. Follow the **README** instructions on the backend and frontend folders for detailed instructions.
6. Implement the missing functionality marked in the code (TODOs) and ensure the app works end-to-end.

Requirements:

1. Data rules:
    - EntityType must be one of the following - Soldier, Tank, Drone, Aircraft, Vehicle, Civilian.
    - TaskForce must be one of the following - Friendly, Enemy.
    - Latitude must be between -90 and 90.
    - Longitude must be between -180 and 180.
    - Entities always belong to exactly one Scenario (no orphan entities).
2. Screens:
    - Scenario List – view all scenarios, navigate to details, create.
    - Create Scenario – the form should contain name and description.
    - Scenario Details – view scenario info and manage its entities.
    - Create Entity – the form must include:
        - EntityType
        - TaskForce
        - Name/Callsign
        - Latitude
        - Longitude
3. Backend
    - The `backend/Backend.Api` project is pre-configured with controllers and Swagger. Build your full backend from there - architecture, domain models, DTOs, and project structure are your design decision.
    - You must implement:
        - Domain models - Scenario, Entity, and any enums required by the data rules.
        - Sata Access Layer - implement `ScenarioRepository` and `EntityRepository` with your chosen storage (in-memory or database).
        - API Controllers - `ScenariosController` and `EntitiesController` with all required endpoints.
        - Server-side filtering - scenarios by name/description, entities by EntityType and TaskForce.
        - Validation - validate all input fields; return proper HTTP status codes and an `ErrorResponse` on 400/404.
4. Frontend - A frontend skeleton is provided (React and Angular) – choose one - complete the TODOs inside it to connect your backend APIs. Follow the README in the chosen folder.
5. Docker:
    - After implementing the core functionality, containerize the solution using Docker:
        i. Create Docker files for Backend and Frontend
        ii. Create a docker-compose.yml.

BAGIRA

iii. Ensure the solution runs with: docker-compose up
iv. Update the main README with Docker instructions

- Notes:
  i. Use multi-stage builds for frontend (build + serve)
  ii. Configure CORS in backend to allow frontend origin
  iii. Use environment variables for API base URL in frontend
  iv. Document any required environment variables
  v. **ONLY if Docker takes more time than expected**, write a short explanation in the README describing how you would containerize the solution and move on.

6. Optional bonus:
   - Connect to a database of your choice (instead of in-memory storage).
   - Global search endpoint (`SearchController`) — search across scenarios and entities by name.
   - Server-side sorting for scenarios and entities.
   - Add tests for validation (backend) or key UI flows (frontend).
   - Implement Update and Delete for Scenarios and Entities end-to-end.
   - UX improvements.

Acceptance criteria:

- User can create a scenario and see it in the scenario list.
- User can open scenario details and view entities belonging to that scenario only.
- User can select TaskForce (Friendly/Enemy) when creating entities, and TaskForce is displayed in entity lists.
- User can filter scenarios by name/description (server-side filtering works).
- User can filter entities by Type and TaskForce (server-side filtering works).
- User can sort scenarios and entities in various fields (server-side sorting works).
- User can search for scenarios and entities using the global search endpoint.
- Backend validates all input fields (coordinates, types, required fields) and returns appropriate error responses.
- Backend returns proper HTTP status codes for all operations.
- Solution runs locally.
- Using clean architecture.
- Code quality and maintainability (Expect this system will have more features in the future).
- Validation and error handling are properly implemented.
- Reasonable scope management within the timebox.
- *** Solution runs in Docker using docker-compose up with all services properly configured and communicating.

Please provide:

- A link to your branch.
- Short notes (bullet points) describing what you completed and any trade-offs.

Good Luck!